

252-0027

Einführung in die Programmierung Übungen

Woche 12: Interfaces, Exception

Joran Bühler

Departement Informatik

ETH Zürich

Ablauf

- **Nachbesprechung Bonus**
- **Interfaces**
- **Errors/exceptions**
- **InstanceOf**
- **Kahoot**

Timed Bonus

Aufgabe 1: ChronoCruiser-Factory (Bonus!)

Für diese Aufgabe sollen Sie ein Rechnungssystem für die Produktionslinie des ChronoCruiser 3000 entwickeln. Je nach Konfiguration des ChronoCruiser 3000 müssen verschiedene Fahrzeugkomponenten verbaut werden. Da verschiedene Komponenten jedoch durch verschiedene Zulieferungsunternehmen produziert werden, müssen sowohl deren Produktionskosten als auch die durch den Einbau entstehende Mehrwertsteuer berechnet werden.

Im bereitgestellten Projekt finden Sie drei Klassen `Cost`, `Part` und `Factory`.

`Cost` ist eine einfache Klasse, welche sowohl die Produktionskosten (Attribut `productionCost`), die Mehrwertsteuer ¹ (Attribut `vat`), als auch die Luxussteuer (Attribut `luxuryTax`, Teil (b)) eines Produkts speichert. Die Attribute sind alle vom Typ `int`.

`Part` ist eine Klasse mit einer Methode `process` (`Cost c`) und dient als Basis für verschiedene Unterklassen. Die Methode `process` modifiziert das gegebene `Cost`-Objekt so, dass die Produktionskosten und Steuern nach der Verarbeitung der Komponente in `c` hinterlegt sind. Jede dieser Unterklassen implementiert die Methode `process` so, dass die Produktionskosten und Steuern verschiedener Bauteile, wie in der folgenden Tabelle beschrieben, berechnet werden.

Timed Bonus

Komponente	Produktionskosten	MWSt
Flügeltüren	2000	3%
Fluxkompensator	Verdopplung der aktuellen Produktionskosten	7%
Schwebeumwandlung	Erhöhung um 20% der aktuellen Produktionskosten	10%
Einklappbare Räder	0 bis 7000: Erhöhung um 7000, aber maximal auf 100000	7%
Outatime-Kennzeichen	100 bis 50000: Erhöhung um 100, aber mind. auf 50000	10%

`Factory` enthält eine statische Methode `computeCost`. Diese Methode erhält ein Array von `Part`-Objekten und gibt ein `Cost`-Objekt zurück, welches die finalen Produktionskosten, die Mehrwertsteuer und (in Teil (b)) die Luxussteuer des Endprodukts speichert. Für die Berechnung dieses Objekts iteriert die Methode über die gegebenen `Part`-Objekte und ruft für jedes Objekt die Methode `process` auf. Die Luxussteuer ist nur relevant für Teil (b).

Aufgaben Bearbeiten Sie nun die folgenden Teilaufgaben:

- a) **Kostenberechnung** Implementieren Sie die `process-Methode` für die gegebenen Part-Unterklassen aus der Tabelle oben, sodass die Produktionskosten und die Mehrwertsteuer der Komponente entsprechend der angegebenen Werte in `Cost` gespeichert werden. Zur Berechnung der Gesamtkosten wird dabei die Methode `Factory.computeCost` verwendet, welche bereits gegeben ist.
- b) **Luxussteuern** Erweitern Sie Ihre Implementierung so, dass die Methode `computeCost` nach Berechnung der gesamten Kosten eine zusätzliche Luxussteuer in `Cost.luxuryTax` speichert. Die Luxussteuer wird nur erhoben, wenn mindestens eine der folgenden zwei neuen Komponenten verbaut wird:

Komponente	Produktionskosten	MWSt	Luxussteuer
First-Edition-Fluxkompensator	wie ein regulärer "Fluxkompensator"		5%
Verchromte Räder	wie reguläre "Einklappbare Räder"		5%

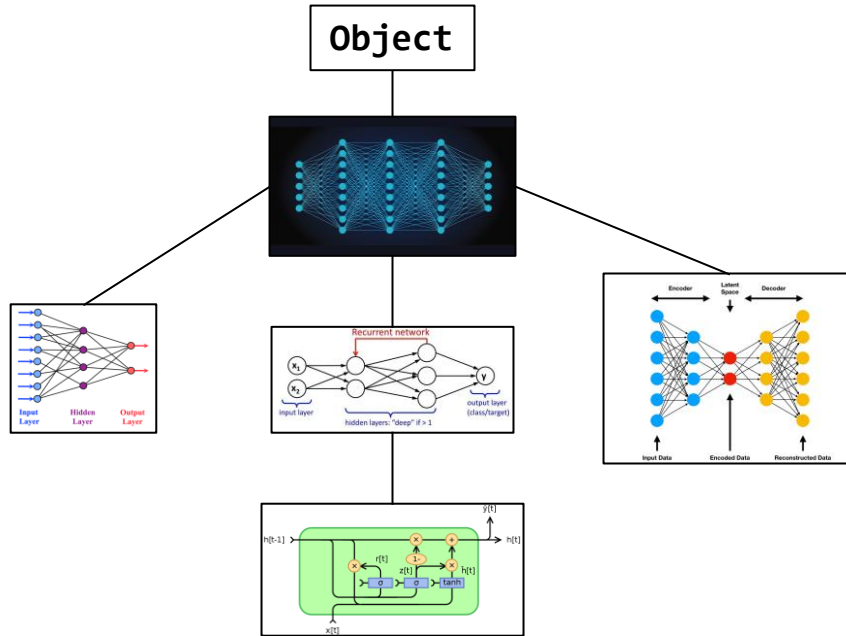
Die Luxussteuer berechnet sich als 5% der gesamten Produktionskosten des Endprodukts, falls mindestens eine der beiden Luxuskomponenten verbaut wurde. Beim Einbau mehrerer Luxuskomponenten wird trotzdem nur eine einmalige Luxussteuer erhoben.

- c) **Konfigurationsfehler** Passen Sie die Methode `computeCost` so an, dass `null` zurückgegeben wird, falls die Liste an Komponenten einen der folgenden Konfigurationsfehler aufweist:
- Es wurde mehr als ein Fluxkompensator verbaut.
 - Es wurde eine Schwebeumwandlung verbaut, nachdem bereits ein Fluxkompensator verbaut wurde.
 - Es wurde ein Outatime-Kennzeichen verbaut, nachdem bereits ein Fluxkompensator verbaut wurde.

Beachten Sie, dass diese Konfigurationsfehler die Luxuskomponenten aus Teil (b) nicht betreffen. Zum Beispiel ist es erlaubt, sowohl einen First-Edition-Fluxkompensator als auch einen regulären Fluxkompensator zu verbauen, nur nicht mehr als einen regulären Fluxkompensator.

Interfaces

Klassen: Neural Network

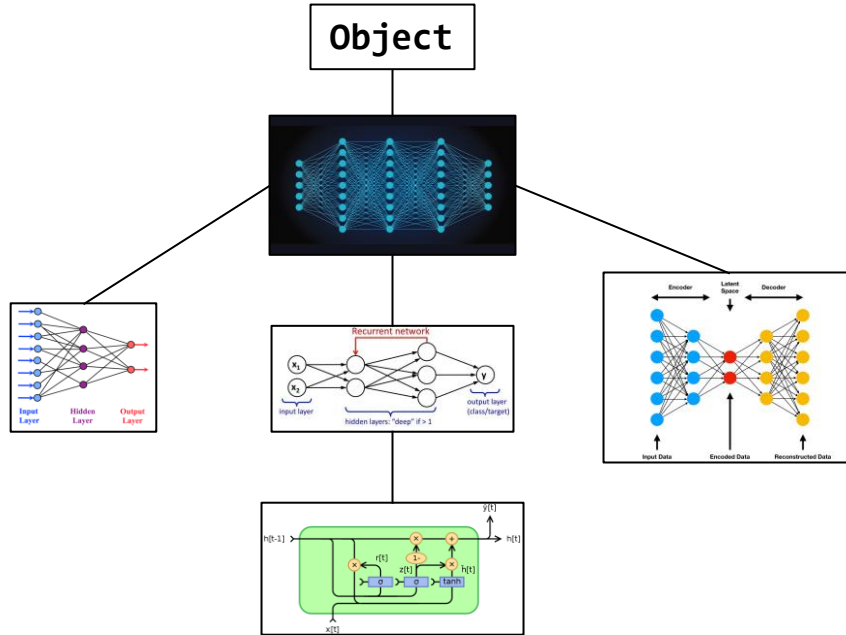


Was macht neuronale Netzwerke aus?

- Nodes, Layers, Weights, etc.
- **train-Methode:** Mit dieser Methode können wir das neuronale Netzwerk trainieren.
- **predict-Methode:** Mit dieser Methode können wir gegebenen Inputs einen Output generieren.

Dafür eignet sich ein Interface!

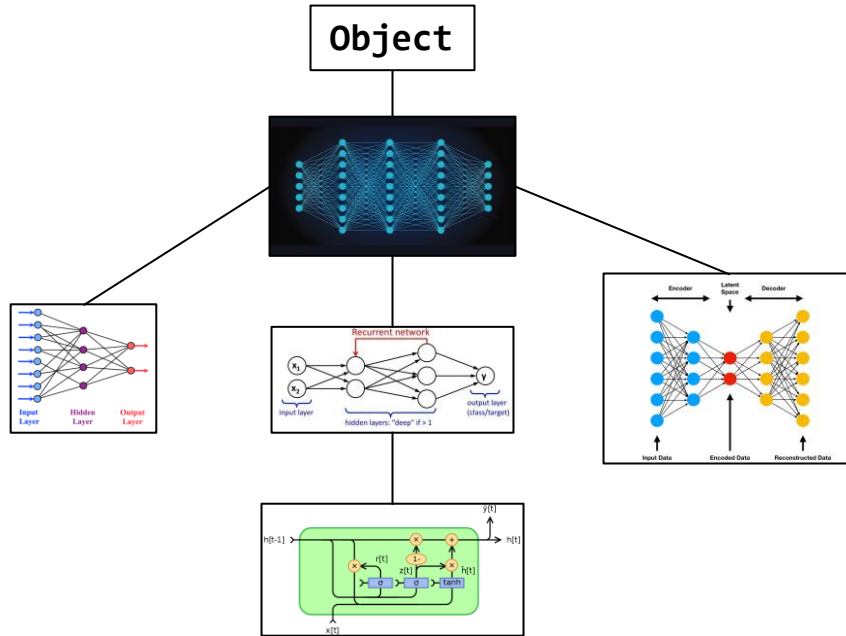
Interfaces: Neural Network



Interfaces:

- Definieren was für ein **Verhalten** eine Klasse haben muss, damit sie das Interface implementiert.
- **Wie** diese Methoden implementiert werden, ist **nicht** Teil des Interfaces.
- Deshalb enthalten Interfaces auch **keine Attribute ausser Konstanten**.
- Jedes Attribut ist public, static und final.

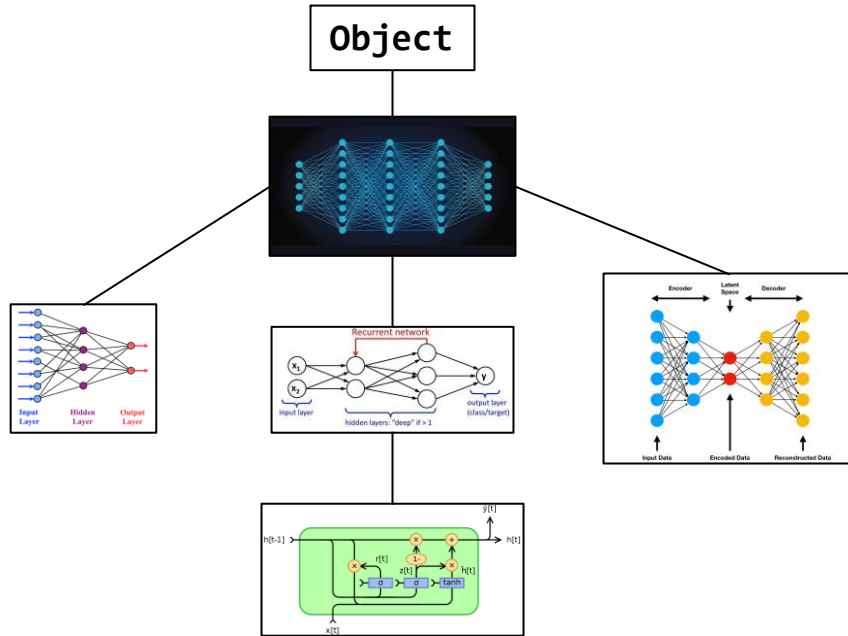
Interfaces: Neural Network



Was macht neuronale Netzwerke aus?

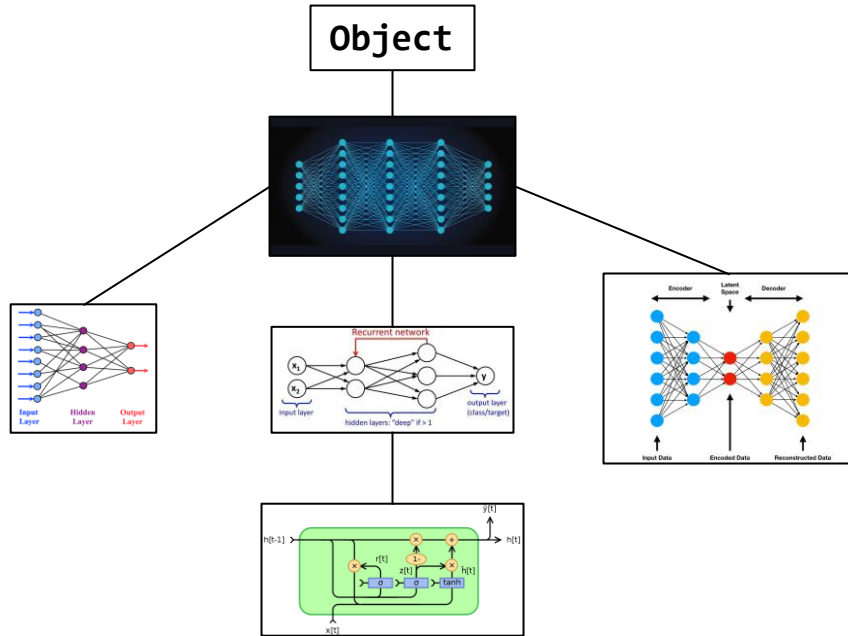
- ~~Nodes, Layers, Weights, etc.~~ **Nicht Teil des Interface.**
- **train-Methode:** Mit dieser Methode können wir das neuronale Netzwerk trainieren.
- **predict-Methode:** Mit dieser Methode können wir gegebenen Inputs einen Output generieren.

Interfaces: Neural Network



```
public interface Neural {
    public void train();
    public void predict(int[] inputs);
}
```

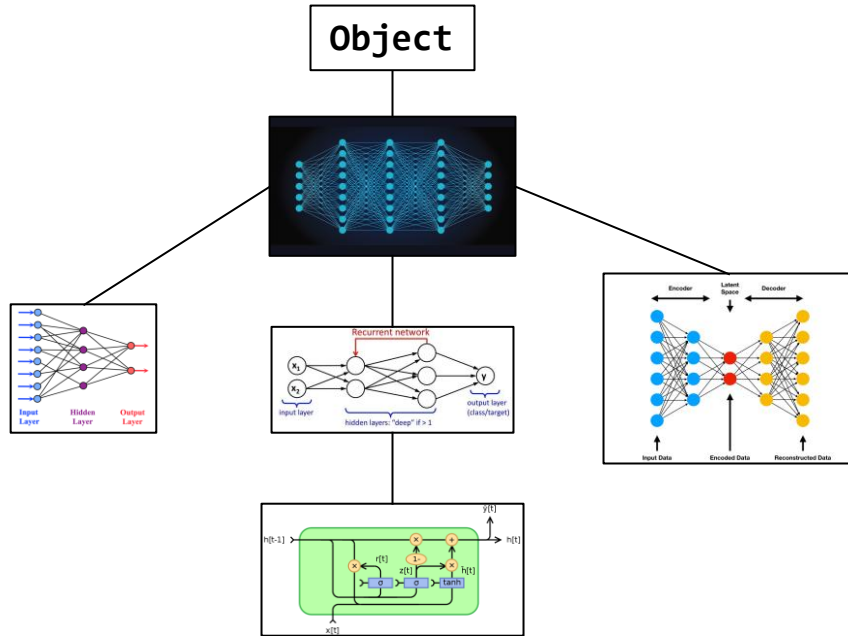
Interfaces: Neural Network



Darf man das?

```
private interface Neural {  
    public void train();  
    public void predict(int[] inputs);  
}
```

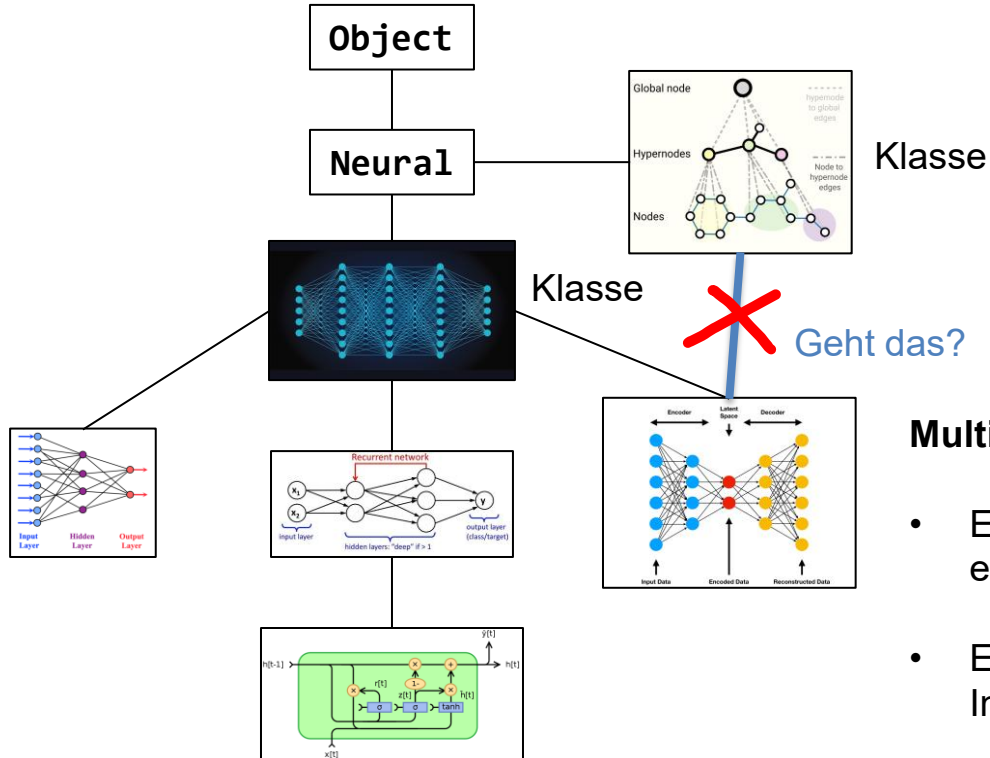
Interfaces: Neural Network



Interfaces müssen einen public oder default modifier haben!

```
private interface Neural {  
    public void train();  
    public void predict(int[] inputs);  
}
```

Interfaces: Neural Network



Interfaces: Inheritance

Inheritance von Interfaces:

- Ein Interface kann von einem anderen Interface erben.
- Erbende Interfaces müssen alle Methoden haben, die das Interface darüber hat.

Interfaces: ArrayList

Module `java.base`

Package `java.util`

Class ArrayList<E>

`java.lang.Object`

`java.util.AbstractCollection<E>`

`java.util.AbstractList<E>`

`java.util.ArrayList<E>`

Type Parameters:

E - the type of elements in this list

All Implemented Interfaces:

`Serializable`, `Cloneable`, `Iterable<E>`, `Collection<E>`, `List<E>`, `RandomAccess`, `SequencedCollection<E>`

ArrayList:

- **Implementiert:** `Serializable`, `Cloneable`, `Iterable`, `Collection`, `List`, ...
- **Extends:** `AbstractList`

Collection Interface:

Module `java.base`

Package `java.util`

Interface `Collection<E>`

Type Parameters:

`E` - the type of elements in this collection

All Superinterfaces:

`Iterable<E>`

All Known Subinterfaces:

`BeanContext`, `BeanContextServices`, `BlockingDeque<E>`, `BlockingQueue<E>`, `Deque<E>`, `EventSet`, `List<E>`, `NavigableSet<E>`, `Queue<E>`, `SequencedCollection<E>`, `SequencedSet<E>`, `Set<E>`, `SortedSet<E>`, `TransferQueue<E>`

All Known Implementing Classes:

`AbstractCollection`, `AbstractList`, `AbstractQueue`, `AbstractSequentialList`, `AbstractSet`, `ArrayBlockingQueue`, `ArrayDeque`, `ArrayList`, `AttributeList`, `BeanContextServicesSupport`, `BeanContextSupport`, `ConcurrentHashMap`, `KeySetView`, `ConcurrentLinkedDeque`, `ConcurrentLinkedQueue`, `ConcurrentSkipListSet`, `CopyOnWriteArrayList`, `CopyOnWriteArraySet`, `DelayQueue`, `EnumSet`, `HashSet`, `JobStateReasons`, `LinkedBlockingDeque`, `LinkedBlockingQueue`, `LinkedHashSet`, `LinkedList`, `LinkedTransferQueue`, `PriorityBlockingQueue`, `PriorityQueue`, `RoleList`, `RoleUnresolvedList`, `Stack`, `SynchronousQueue`, `TreeSet`, `Vector`

Modifier and Type	Method	Description
<code>boolean</code>	<code>add(E e)</code>	Ensures that this collection contains the specified element (optional operation).
<code>boolean</code>	<code>addAll(Collection<? extends E> c)</code>	Adds all of the elements in the specified collection to this collection (optional operation).
<code>void</code>	<code>clear()</code>	Removes all of the elements from this collection (optional operation).
<code>boolean</code>	<code>contains(Object o)</code>	Returns <code>true</code> if this collection contains the specified element.
<code>boolean</code>	<code>containsAll(Collection<?> c)</code>	Returns <code>true</code> if this collection contains all of the elements in the specified collection.
<code>boolean</code>	<code>equals(Object o)</code>	Compares the specified object with this collection for equality.

Summary (geeksforgeeks)

Features	Class	Interface
Keyword	The keyword used to create a class is "class".	The keyword used to create an interface is "interface".
Instantiation	A class can be instantiated, i.e., objects of a class can be created.	An interface cannot be instantiated directly, instead, it is implemented by a class or a struct.
Inheritance	Classes do not support multiple inheritance.	Interface supports multiple inheritance.
Inheritance Mechanism	A class can inherit another class using the keyword extends.	A class uses implements to adopt an interface, while an interface uses extends to inherit another interface.
Constructors	It can contain constructors.	It cannot contain constructors.

Access Specifiers	Variables and methods in a class can be declared using any access specifier(public, private, default, protected).	All variables and methods in an interface are declared public
Variables	Variables in a class can be static, final, or neither.	All variables are static and final.
Purpose	A class is a blueprint for creating objects and encapsulates data and behaviour.	An interface specifies a contract for classes to implement by focusing on capabilities rather than implementation.

Compiler-Fehler vs Exceptions

Compile Fehler:

- Syntax Fehler
 - Keine Klammern, Semikolons vergessen?
 - Existiert eine Methode mit diesem Namen und dieser Signatur?
- Typenkompatibilität
 - Compiler-Brille
 - `int i = 4.5;`
 - **Präzisionsverlust:** Der Compiler beschwert sich, explizites Casting erforderlich.
 - `String s = (String) new Integer(42);`
 - **Kein Vererbungsverhältnis:** Ein Cast zu Laufzeit würde nie funktionieren.

Exceptions:

- Wir überprüfen das Programm auf Logikfehler
 - **ArithmeticException**: Tritt auf bei fehlerhaften mathematischen Operationen (z.B. Division durch Null).
 - **NullPointerException**: Entsteht, wenn auf ein *null*-Objekt zugegriffen wird.
 - **ClassCastException**: Wird geworfen bei einem ungültigen Cast zwischen inkompatiblen Objekten.
 - **ArrayIndexOutOfBoundsException**: Passiert, wenn auf einen ungültigen Index eines Arrays zugegriffen wird.
 - **NumberFormatException**: Tritt auf, wenn versucht wird, einen String in eine Zahl umzuwandeln, der kein korrektes Zahlenformat hat.

Beispiel 1



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int a = 0;  
4         int b = 5;  
5         System.out.println((a > 0) && (b / a > 1));  
6     }  
7 }
```

Beispiel 1 – Das sieht der Compiler

```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int a = int;  
4         int b = int;  
5         System.out.println((int > int) && (int / int > int));  
6     }  
7 }
```


Beispiel 1



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int a = 0;  
4         int b = 5;  
5         System.out.println((a > 0) && (b / a > 1));  
6     }  
7 }
```

A red bracket is drawn under the expression `(a > 0)` in line 5, with the word **False** written in red text below it.

False

Beispiel 1 – Ohne Short-Circuiting



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int a = 0;  
4         int b = 5;  
5         System.out.println((b != 0) && (b / a > 1));  
6     }  
7 }
```

True Exception

Aufgabe 1



```
1 int x = 10
2 double y = 5.0;
3 int z = x + y;
```

Compile-Fehler



Exception



Aufgabe 2



```
1 int x = 10;  
2 int y = 5;  
3 double result = (double x + y;
```

Compile-Fehler



Exception



Aufgabe 3



```
1 String num = "123";  
2 int number = (int) num;  
3 double result = 2 * number;
```

Compile-Fehler



Exception



Aufgabe 4



```
1 double x = 10.5;  
2 int y = (int) (x * "2");  
3 int z = x
```

Compile-Fehler



Exception



Aufgabe 5



```
1 Integer a = null;  
2 int b = 5;  
3 int result = a + b;
```

Compile-Fehler



Exception



Try - Catch

```
try {  
    fillArray();  
} catch (ArrayIndexOutOfBoundsException e) {  
    //make the array Longer  
} catch (Exception e) {  
    //Oh no, there is another exception.  
}
```


Eigene Exceptions:



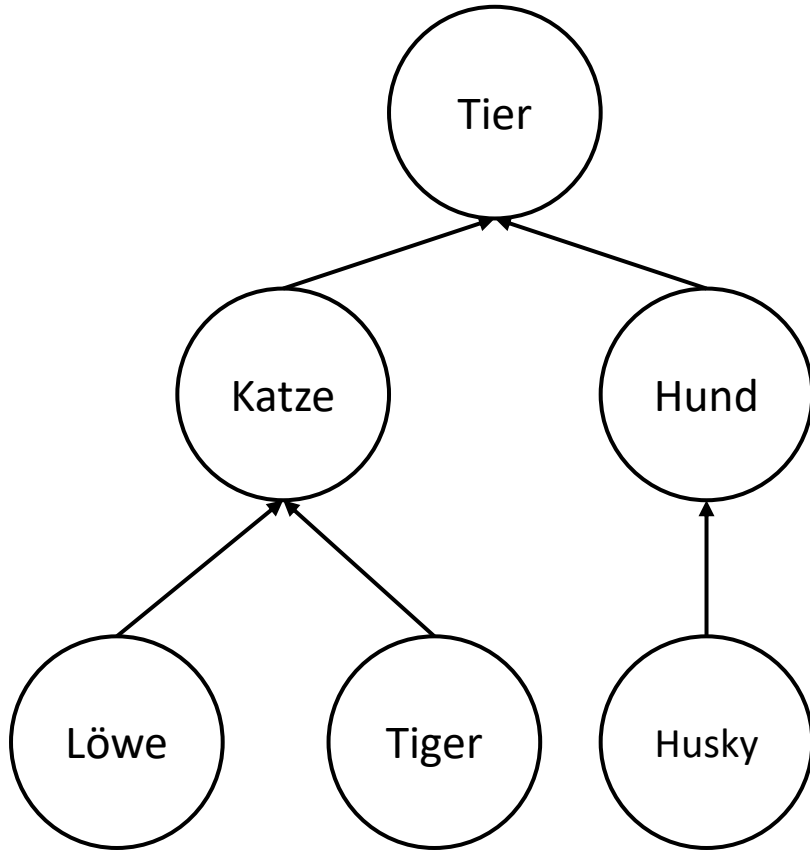
```
1 // Eigene Exception
2 class OverheatedException extends Exception {
3     public OverheatedException(String message) {
4         super(message);
5     }
6 }
7
8 class Machine {
9     private int temperature;
10
11     public Machine(int temperature) {
12         this.temperature = temperature;
13     }
14
15     public void checkTemperature() throws OverheatedException {
16         if (temperature > 100) {
17             throw new OverheatedException("Machine is overheated!");
18         }
19     }
20 }
```

instanceof

konkretesObjekt instanceof Klasse

- Prüft, ob der **dynamische** Typ von **konkretesObjekt** eine Unterklasse von **Klasse** ist oder **Klasse** selbst und gibt **true** zurück, falls dies der Fall ist.
- Wenn **konkretesObjekt null** ist, gibt **instanceof** immer **false** zurück.
- wenn statischer Typ des Objekts und zu prüfende Klasse **keine gemeinsame Vererbungshierarchie** haben, erkennt Compiler, dass Prüfung sinnlos ist, und gibt einen Compile-Fehler zurück.
- Funktioniert auch mit **Interfaces**

instanceof



true oder false?

```
Tier hd = new Hund();
```

```
hd instanceof Husky
```

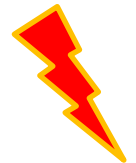


```
hd instanceof Tier
```



```
Hund h2 = new Hund();
```

```
h2 instanceof Katze
```



Statischer Typ
Dynamischer Typ

Kahoot

- <https://create.kahoot.it/details/961f3431-853a-47a9-b12e-52fc51f7eb48>

Nächste Woche:

Alte Theorieprüfung Lösen

- Oder programmierprüfung?

Slides durchgehen (Collections)

Nachbesprechung Loopinvarianten

Aufgabe 1: Loop- Invarianten

1. Um die Loop-Invariante einfacher schreiben zu können, dürfen Sie `min(arr, i)` benutzen. Hier steht `min(arr, i)` für das minimale Element im Array `arr` von Index 0 bis Index `i` (exklusiv). Alternativ könnte man auch formale Notation benutzen, in dem man mit Quantoren arbeitet. Zum Beispiel, falls `m == min(arr, i)`, dann könnten Sie äquivalent Folgendes schreiben

$$\forall 0 \leq j < i \ (arr[j] \leq m)$$

```
int min(int[] arr) {  
    // Precondition: arr != null && 0 < arr.length  
    int m = arr[0];  
    int i = 1;  
  
    // Loop-Invariante:  
    while (i < arr.length) {  
        if (arr[i] < m) {  
            m = arr[i];  
        }  
  
        i++;  
    }  
  
    // Postcondition: m = min(arr, arr.length)  
    return m;  
}
```

Official Solution: `0 <= i <= arr.length && m = min(arr, i) && arr != null && 0 < arr.length`